

Smart Robot Simulation System

Submitted By

Student Name	Student ID
Nawyaz Ahammed Hridoy	251-15-139

LAB PROJECT REPORT

This Report is Presented in Partial Fulfillment of the course
**CSE222 : Object Oriented Programming LAB in the Department of
Computer Science and Engineering**



DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

9 August 2026

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Subarna Akter Liza, Lecturer** , Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

Subarna Akter Liza

Lecturer

Department of Computer Science and Engineering
Daffodil International University

Submitted by

Nawyaz Ahammed
Hridoy
Student ID:251-15-139
Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The course incorporates the following course outcomes:

Table 1: Course Outcome Statements

CO's	Statements
CO1	Understand the core principles of Object-Oriented Programming (Encapsulation, Abstraction, Inheritance, Polymorphism).
CO2	Design and develop Java applications by modeling real-world entities into software objects.
CO3	Analyze simulated environments (such as robot movement and collision) to implement algorithmic problem-solving.
CO4	Utilize modern Java development tools (IDEs) to write, debug, and execute object-oriented code.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Learning Domains	Knowledge Profile	Complex Engineering Problem	Complex Engineering Activities
CO1	PO1	Cognitive (Understand)	KP3	EP1	EA1
CO2	PO3	Psychomotor (Create)	KP5	EP3	EA2
CO3	PO2	Cognitive (Analyze)	KP4	EP2	EA1
CO4	PO5	Psychomotor (Apply)	KP6	EP6	EA3

The mapping justification of this table is provided in section 4.3.1, 4.3.2 and 4.3.3.

Table of Contents

Declaration	2
Course & Program Outcome	3
1 Introduction	4
1.1 Introduction	4
1.2 Motivation	5
1.3 Objectives	5
1.4 Feasibility Study	5
1.5 Gap Analysis	5
1.6 Project Outcome	6
2 System Architecture	7
2.1 Requirement Analysis & Design Specification	7
2.1.1 Requirements	7
2.1.2 ERD Diagram	7
2.2 Use of Modern Tools	8
3 Implementation and Results	9
3.1 Implementation	9
3.2 Output	12
3.3 Discussion	12
4 Engineering Standards and Mapping	13
4.1 Impact on Society, Environment and Sustainability	13
4.1.1 Impact on Life	13
4.1.2 Impact on Society & Environment	13
4.1.3 Ethical Aspects	13
4.1.4 Sustainability Plan	14
4.2 Complex Engineering Problem	14
4.2.1 Mapping of Program Outcome	14
4.2.2 Complex Problem Solving	15
4.2.3 Complex Engineering Activities	16
5 Conclusion	17
5.1 Summary	17
5.2 Limitation	17
5.3 Future Work	18
References	19
Source Code	19

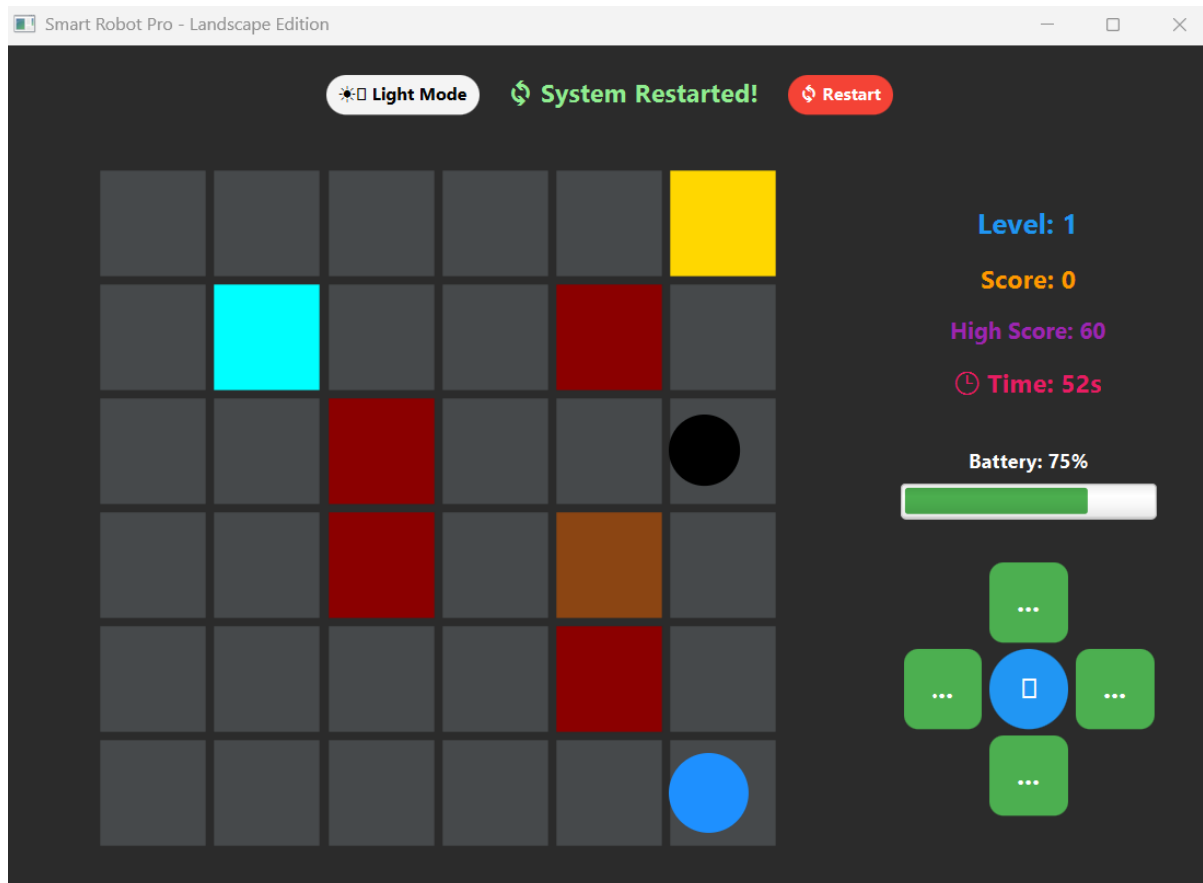
Chapter 1

Introduction

This chapter introduces the background, motivation, and core objectives of the Smart Robot Simulation System. It also evaluates the project's feasibility, identifies gaps in current simulation models, and outlines the final expected outcomes of the software.

1.1 Introduction

The Smart Robot Simulation System is a 2D mini-game built with JavaFX. It models how a robot acts inside a virtual grid. We made this project to solve a common problem. Many students learn Object-Oriented Programming (OOP) in class, but they struggle to see how it works in real life. Our project solves this issue. It shows exactly how OOP concepts handle real-time actions like movement, battery drain, and wall collisions on a screen.



1.2 Motivation

We wanted to connect our textbook OOP theories with a fun and working project. Making a robot simulator helps us practice writing clean and modular code. It teaches us how different objects, like robots, walls, and batteries, talk to each other without breaking the system. Building this project greatly improved our logical thinking and coding skills.

1.3 Objectives

Our main goals for this project are:

- To use Encapsulation to protect sensitive data, like the robot's battery level and map coordinates.
- To use Abstraction to create hidden blueprints for our simulation objects.
- To use Inheritance to share common code between basic robots and advanced cleaning robots.
- To use Polymorphism to let different objects respond to the same commands in their own unique ways.
- To build a fully interactive JavaFX interface with an AI enemy, power-ups, and a file-based high-score system.

1.4 Feasibility Study

Big robotic simulators in the industry, like ROS, are very heavy and complex. They need high-end computers to run. Our project is a lightweight and simple alternative. It runs smoothly on any basic computer that has Java installed. We only used standard Java and JavaFX libraries. Because of this, it was very practical to finish within our lab timeframe.

1.5 Gap Analysis

Many learning tools hide the background code from the user. This stops students from learning the real architecture. On the other hand, simple coding tasks often look boring and lack real-world context. Our project fills this exact gap. We built a fully visible system where every line of Java code directly controls a physical action on the screen.

1.6 Project Outcome

The final result is a complete, graphical desktop application. Users can control a smart robot to clean dirt while dodging a smart AI enemy. The game automatically changes levels, handles power-up shields, and saves the highest score using File I/O. Most importantly, it successfully proves the four main pillars of OOP in a real working environment.

Chapter 2

System Architecture

This chapter explains the technical requirements and the design of our Smart Robot Simulation System. It also shows how our main classes connect and the modern tools we used to build the game.

2.1 Requirement Analysis & Design Specification

We built this system using a modular, object-oriented approach. Every physical part of the game, like the robot or a wall, is a separate software object. This design keeps the code clean, and it makes adding new features very easy.

2.1.1 Requirements

To develop and play this simulation, you need the following setup:

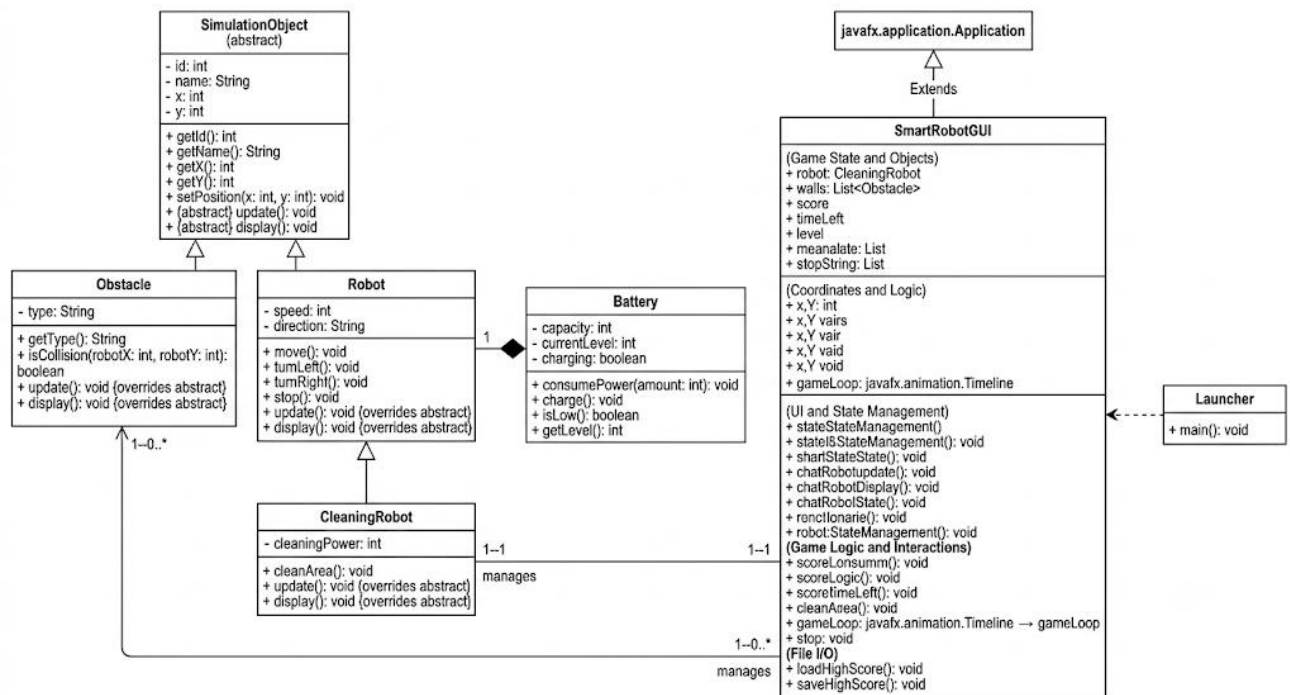
- **Programming Language:** Java (JDK 8 or higher) with JavaFX support.
- **Development Environment:** IntelliJ IDEA or Apache NetBeans.
- **Operating System:** Windows.
- **Hardware Requirements:** A standard computer with at least 4GB of RAM to run the game smoothly.

2.1.2 System Design / Class Diagram

Our system follows a strict OOP structure. Here is how the core classes are connected:

- **SimulationObject:** This is our abstract parent class. It holds the basic blueprint, like the ID, Name, and X/Y coordinates.
- **Robot:** This class inherits from `SimulationObject`. It handles the movement and battery logic.
- **Obstacle:** This class also inherits from `SimulationObject`. It manages the walls for collision detection.

- **CleaningRobot:** This class inherits from the `Robot` class. It adds special cleaning powers. This is a great example of multilevel inheritance.
- **SmartRobotGUI:** This is our main JavaFX class. It handles the graphical window, the game loop, and user button clicks.



2.2 Use of Modern Tools

We used modern software engineering tools to make sure our code is efficient and error-free:

- **Java Development Kit (JDK):** This provides the core compiler to run our object-oriented logic.
- **JavaFX:** We used this graphics library to build the 2D visual interface. It replaced the basic command-line console.
- **IntelliJ IDEA (IDE):** This was our primary coding tool. It helped us find syntax errors in real-time and debug the game easily.
- **File I/O Utility:** We used Java's built-in `FileWriter` and `Scanner` classes to read and save the high score in a text file.

Chapter 3

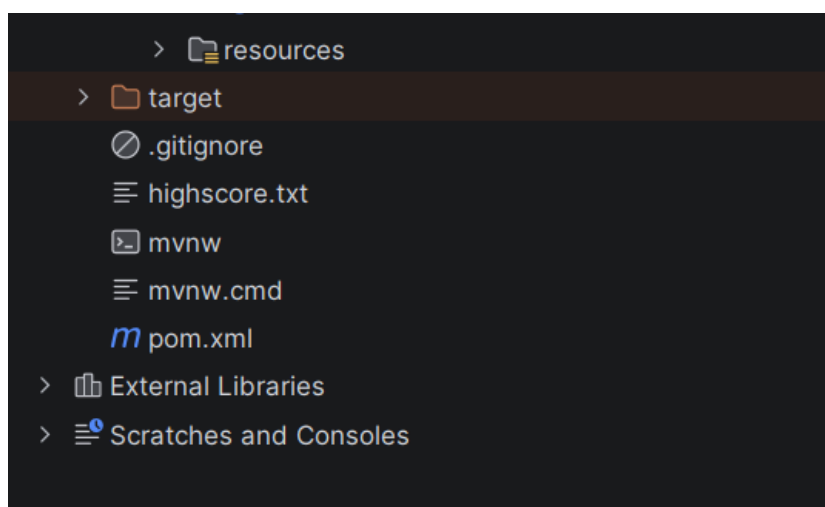
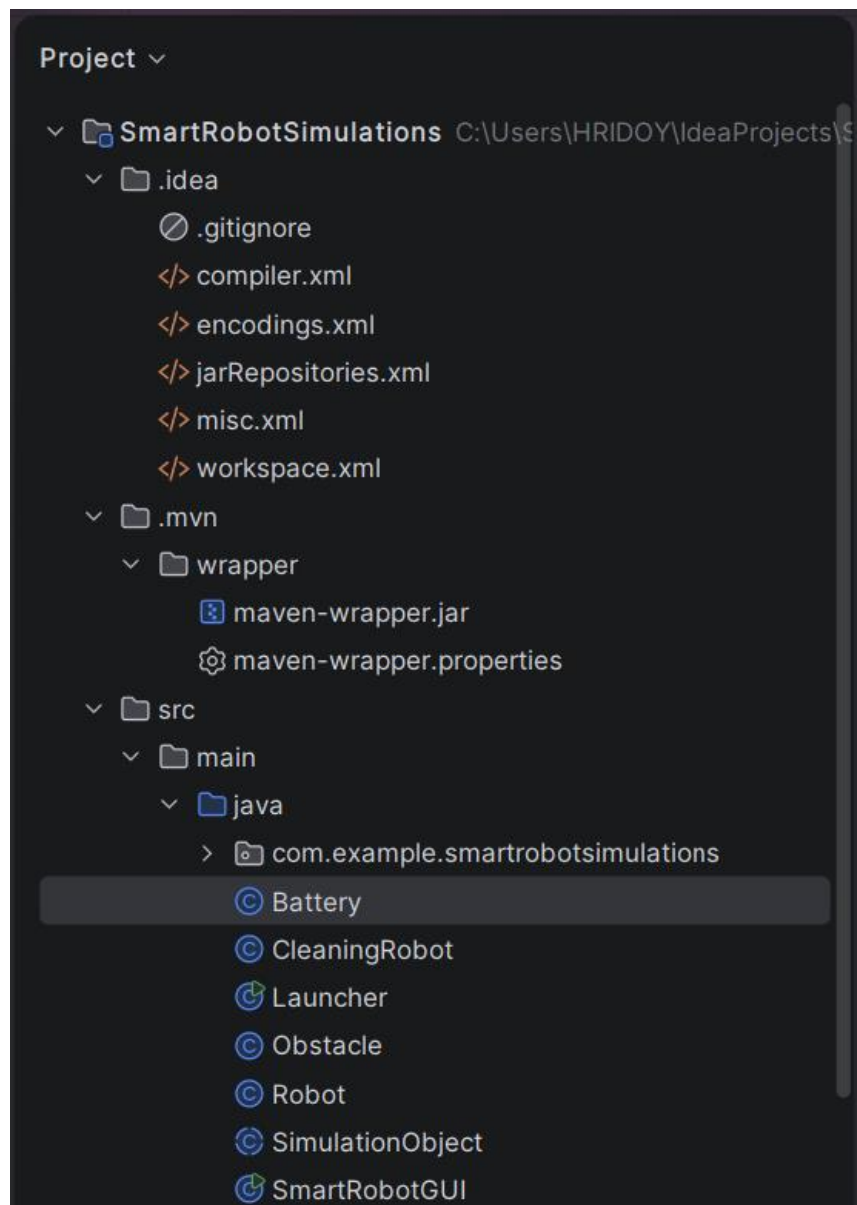
Implementation and Results

This chapter explains how we turned our designs into a real working application. It shows how the Object-Oriented principles are used in the actual Java code. It also includes screenshots of the final game and discusses how the system performs.

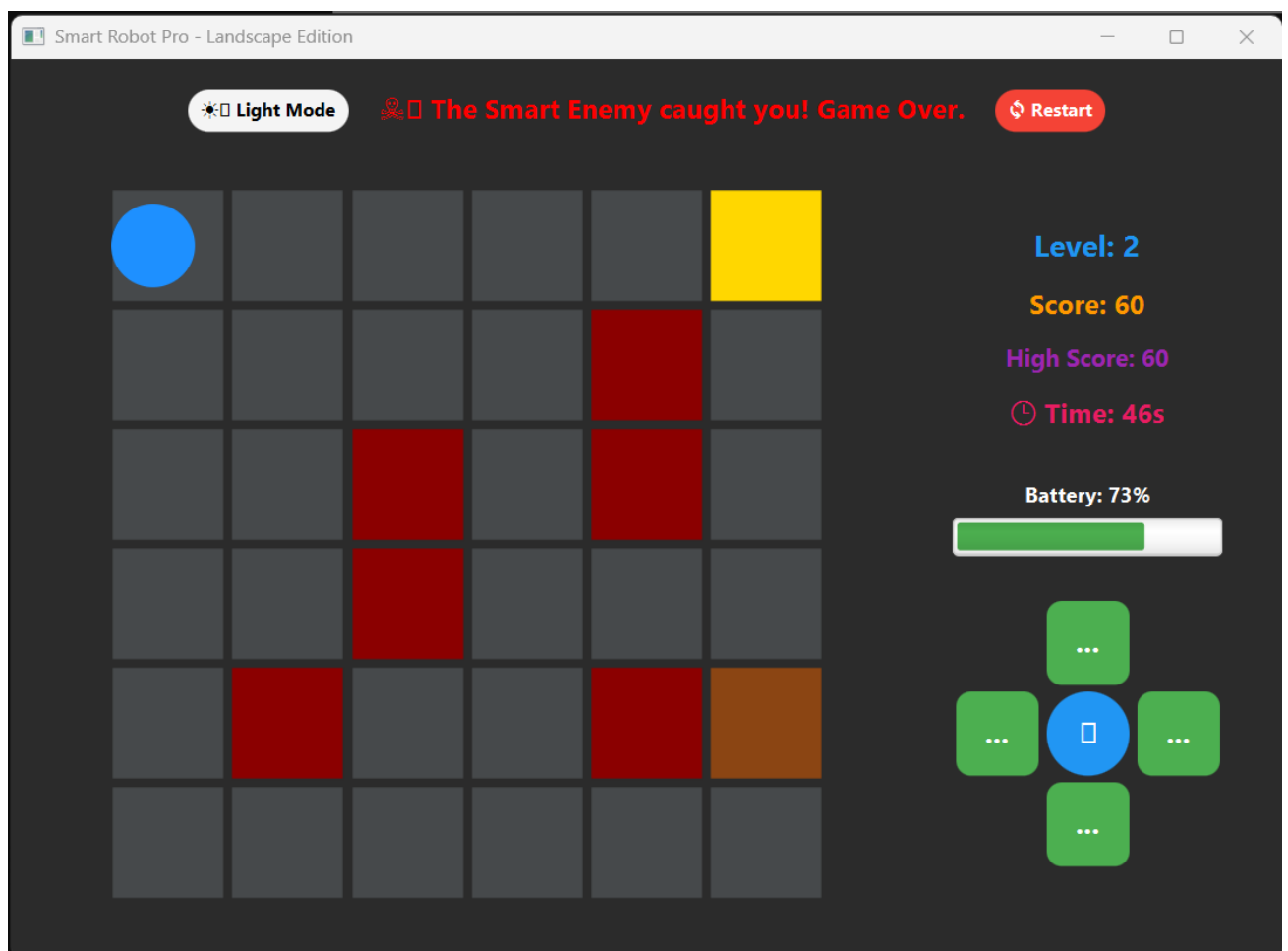
3.1 Implementation

We translated our system design into a functional JavaFX application. The core logic relies strictly on the four main pillars of OOP. This makes the code organized and easy to manage.

- **Encapsulation:** We protect vital data using private access modifiers. In our code, the `Battery` class encapsulates `currentLevel`, which can only be modified through the `consumePower()` method. Also, the robot's coordinates (`x`, `y`) are updated only via the `setPosition()` method. This keeps our game states secure.
- **Abstraction:** We use `SimulationObject` as an abstract base class. It defines signatures like `update()` and `display()`. This forces child classes to implement their own logic, like drawing the robot visual or setting up a wall. It hides the backend details from the main game loop.
- **Inheritance:** We share common code by extending classes. The `Robot` and `Obstacle` classes inherit basics like `ID`, `Name`, `X`, and `Y` from `SimulationObject`. Also, `CleaningRobot` extends the `Robot` class. This inherits movement mechanics and adds cleaning functions, showing multilevel inheritance.
- **Polymorphism:** We use method overriding extensively. The main grid reroofing logic calls `.display()` on every object type. The Java runtime automatically figures out whether it needs to draw a gray empty box, a red wall, a brown dirt spot, a cyan power-up, or the blue robot circular icon.



We implemented the advanced features in the `SmartRobotGUI` class using a game loop (`Timeline`). It runs every second to check all game rules. The level logic checks the score, automatically advancing to Level 2 and adding new walls when the user hits 50 points. The smart enemy uses an tracking algorithm to constantly compare its position with the user's position to move closer. Power-up boxes appear randomly, and collecting one triggers the 10-second shield timer, which stops battery drain and protects against enemy collisions.



3.2 Output

When the simulation starts, the `Launcher` class executes `SmartRobotGUI`, opening a graphical window (850x600 resolution). You see the 6x6 grid map on the left and the control panel on the right.

The user controls the robot by clicking the green D-Pad directional buttons on the right. You can see the Battery Progress Bar decrease with every step, showcasing energy drain. Clicking the central 'Clean' () button when on a brown spot removes the dirt, increases the score, and plays a beep sound. If the user hits a red wall, a collision alert appears, and a system beep sounds. The game loop automatically spawns the cyan power-up boxes; grabbing one turns your robot icon cyan.

3.3 Discussion

The visual results show that the simulation engine logic is working perfectly. The interactive GUI accurately captures user button clicks and updates the map instantly. We verified that Encapsulation is functional because when the robot moves, the battery progress bar visibly lowers, proving the internal state is changing safely. Polymorphism is demonstrated clearly on the grid; different objects like charge points, dirt, power-ups, walls, and robots all render with distinct colors and shapes within the same loop.

Most importantly, the collision detection logic is very accurate. It successfully prevents the robot from moving into solid walls. The game loop immediately captures enemy collisions, stops the timer, and shows the game-over screen, preventing illegal overlapping states.

Chapter 4

Engineering Standards and Mapping

This chapter talks about the engineering standards we used while making this project. We also show how our project work connects to our Course Outcomes (COs) and Program Outcomes (POs). This explains how the system meets the university's academic and professional standards.

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

Our project creates a completely safe virtual environment. It is a great testing ground for understanding robotic logic and OOP principles. Students can learn without actually touching expensive physical hardware. We simulate all movement and collisions digitally. This improves the learning experience for engineering students because it removes the safety risks, like electrical shorts or crashes, that often happen during early hardware testing.

4.1.2 Impact on Society & Environment

Building robotic simulators programmatically is very good for the environment because it reduces e-waste. Developers can test and refine their robot's logic in a purely digital world first. This means we don't need to throw away expensive broken microcontrollers, sensors, or batteries that often fail during real-world trial-and-error testing.

4.1.3 Ethical Aspects

We designed this simulation with full transparency in mind. We defined the Object-Oriented structure clearly, and the code has absolutely no hidden or malicious routines. The rules for battery drain and enemy movement are fair. They work exactly the same regardless of what user inputs are given. This creates a predictable and unbiased system that aligns with standard software engineering ethics.

4.1.4 Sustainability Plan

The project is highly sustainable because we used strict Object-Oriented rules. We achieved high modularity through Abstraction and Inheritance. Future developers can easily add new entities, like different robot types or charging stations, by simply extending our `SimulationObject` class without breaking the existing codebase.

4.2 Complex Engineering Problem

4.2.1 Mapping of Program Outcome

In this section, provide a mapping of the problem and provided solution with targeted Program Outcomes (PO's).

Table 4.1: Justification of Program Outcomes

POs	Justification of Mapping (Project Perspective)
PO1	Applied foundational Object-Oriented Programming theories to structure the Smart Robot Simulation system using Java classes and objects.
PO2	Analyzed movement mechanics and boundary conditions to accurately formulate the collision detection logic between the Robot and Obstacle objects on a 2D grid.
PO3	Designed a scalable system architecture utilizing abstract classes (<code>SimulationObject</code>) and hierarchical inheritance to meet the required project specifications.
PO5	Leveraged modern Java Integrated Development Environments (IDEs) and built-in utilities (like <code>java.util.Scanner</code>) for writing, compiling, and debugging the simulation program.

4.2.2 Complex Problem Solving

Table 4.2: Mapping with complex problem solving.

EP1 Dept of Knowledge	EP2 Range of Conflicting Requireme nts	EP3 Depth of Analysis	EP4 Familiarity of Issues	EP5 Extent of Applicable Codes	EP6 Extent Of Stakeholde r Involveme nt	EP7 Inter- dependence
✓	✓	✓			✓	

Justification of the mapping :

- **EP1 (Depth of Knowledge):** The project demands a profound understanding of core Object-Oriented Programming (OOP) concepts—such as Encapsulation, Inheritance, Polymorphism, and Abstraction—and Java syntax to successfully build the simulation engine.
- **EP2 (Range of Conflicting Requirements):** The project encounters a low range of conflicting requirements, as the scope is strictly defined by the academic guidelines and focuses entirely on demonstrating software design rather than balancing competing real-world physical or financial constraints.
- **EP3 (Depth of Analysis):** A high depth of analysis is required to develop the logic for grid-based movement, state tracking, battery consumption algorithms, and accurate spatial coordinate comparison for collision detection.
- **EP6 (Extent of Stakeholder Involvement):** Stakeholder involvement is minimal, as the project is developed primarily for academic evaluation by the course supervisor, Subarna Akter Liza , and does not involve diverse external client groups, end-users, or community parties.

4.2.3 Complex Engineering Activities

Table 4.2: Mapping with complex engineering activities.

EA1 Range of resources	EA2 Level of interaction	EA3 Innovation	EA4 Consequences for society and the environment	EA5 Familiarity
✓	✓	✓		

Justification of the mapping (EA1 - EA3):

- **EA1 (Range of resources):** The project utilized a diverse range of software engineering resources, including the Java Development Kit (JDK), modern Integrated Development Environments (IntelliJ IDEA / Apache NetBeans), and foundational UML (Unified Modeling Language) design patterns to structure and execute the simulation.
- **EA2 (Level of interaction):** The level of interaction with external or conflicting technical issues is relatively low. The project is an isolated, academic software model, though it does require carefully resolving the logical interactions and data passing between distinct software objects (such as the Robot, Battery, and Obstacle classes).
- **EA3 (Innovation):** The project demonstrates creative innovation by successfully modeling physical, real-world robotic behaviors—such as spatial movement, battery consumption, and solid boundary collision—using a purely text-based, Object-Oriented software architecture.

Chapter 5

Conclusion

This final chapter summarizes what we achieved with the Smart Robot Simulation System. We also look at the project's current limits and propose ideas for future improvements.

5.1 Summary

Our Smart Robot Simulation System successfully teaches foundational software engineering concepts. We did this by modeling a physical robot inside a virtual world. We built the game with JavaFX and followed the four main rules of Object-Oriented Programming (OOP) very closely. Encapsulation secures the battery and position data. Abstraction creates hidden blueprints for our objects. Inheritance handles common code between general and special robots. Polymorphism creates unique behaviors, like updating the robot visual when the shield is on.

The final product is a graphical desktop game. A player can control a robot to clean dirt while dodging a smart AI enemy. The system handles levels, shield power-ups, and uses File I/O to save the highest score. It successfully proves that OOP is extremely useful in real-world software design.

5.2 Limitations

Even though we met our main goals, the current game has a few limitations:

- **Static Pathfinding:** The robot only moves when the user clicks a button. It has no brain of its own to navigate to a target. If you click a direct path into a wall, the robot just stops; it cannot figure out how to drive around it.
- **Single-Directional Movement:** Right now, the robot can only drive in one direction at a time. It cannot turn its body, drive backwards, or slide diagonally. This makes the movement very stiff and unrealistic.
- **Small Environment:** The 6x6 grid is very small. This limits the player's space to dodge the enemy and makes the shield power-up almost too good.

5.3 Future Work

To make the game better and overcome these current limits, we have several ideas for future updates:

- **Autonomous Navigation (AI):** We can add smart pathfinding algorithms like A* or Dijkstra's. This would let the user just click a target coordinate, and the robot will drive there automatically while navigating around any walls on its own.
- **Expanded Environment Variables:** We can make the grid dynamic. In a bigger map, we can add moving obstacles, extra robots that compete for dirt, or physical charging stations that the player needs to physically drive onto.
- **Better Controls:** Instead of mouse-clicking on-screen buttons, we can add keyboard support (Arrow Keys or W,A,S,D). This would make the movement much faster and much more satisfying to play

References

- [1] Jon Kleinberg and Eva Tardos. *Algorithm design*. Pearson Education India, 2006.
- [2] Herbert Schildt. *Java: The Complete Reference*. McGraw-Hill Education, 12th Edition, 2021. (*This covers all the core OOP concepts like Encapsulation, Inheritance, and Polymorphism used in the project*)
- [3] Paul Deitel and Harvey Deitel. *Java How to Program, Early Objects*. Pearson, 11th Edition, 2017. (*This book is excellent for JavaFX GUI programming and event handling*)
- [4] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th Edition, 2020. (*This reference perfectly justifies the "Smart AI" tracking logic used for the enemy robot*)

Source Code



SmartRobotSimulations.zip